

Homework 1

Problem 4

- Figure 1 shows the errors at each state for an individual search. Setting $d = 0.99$ yields runs where 17 of 30 searches have 0 error (Figure 2). A decay value closer to 1 extends the cooling process, letting simulated annealing step through more states and reduce the errors for the solution.

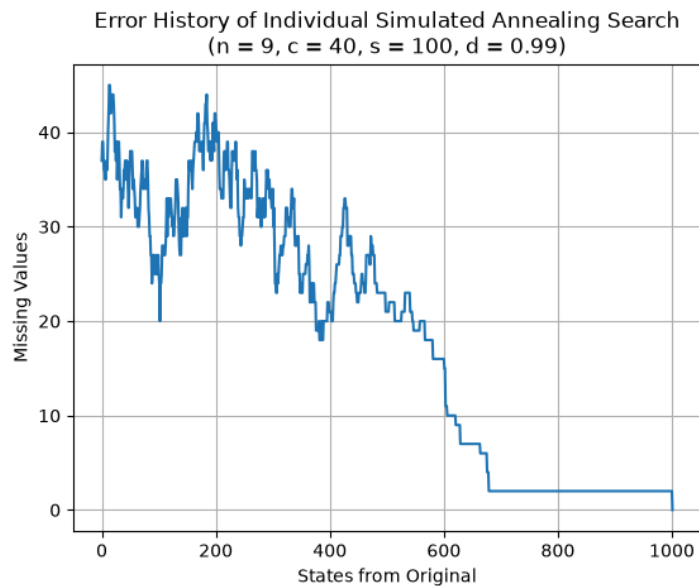


Figure 1.

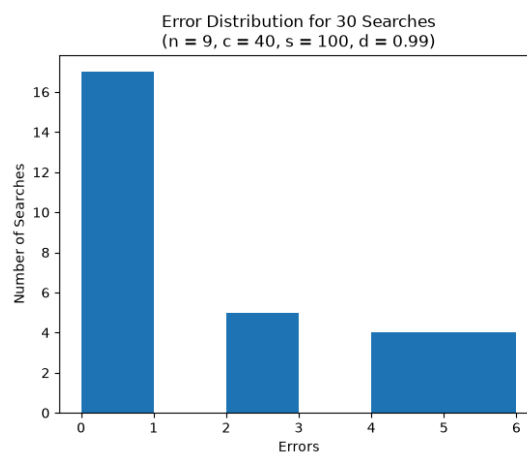


Figure 2.

2. See Figure 3 for the plots with starting temperatures of 1, 10, and 100 respectively. A higher starting temperature means that a search visits more states before the temperature falls below the minimum threshold, which is reflected in the x-axis of each graph. Higher initial temperature also increases the probability of moving to a worse successor, so simulate annealing is more likely to initially explore states with a higher number of errors. This tendency appears in Figure 3 as more initial oscillations when starting temperature increases.

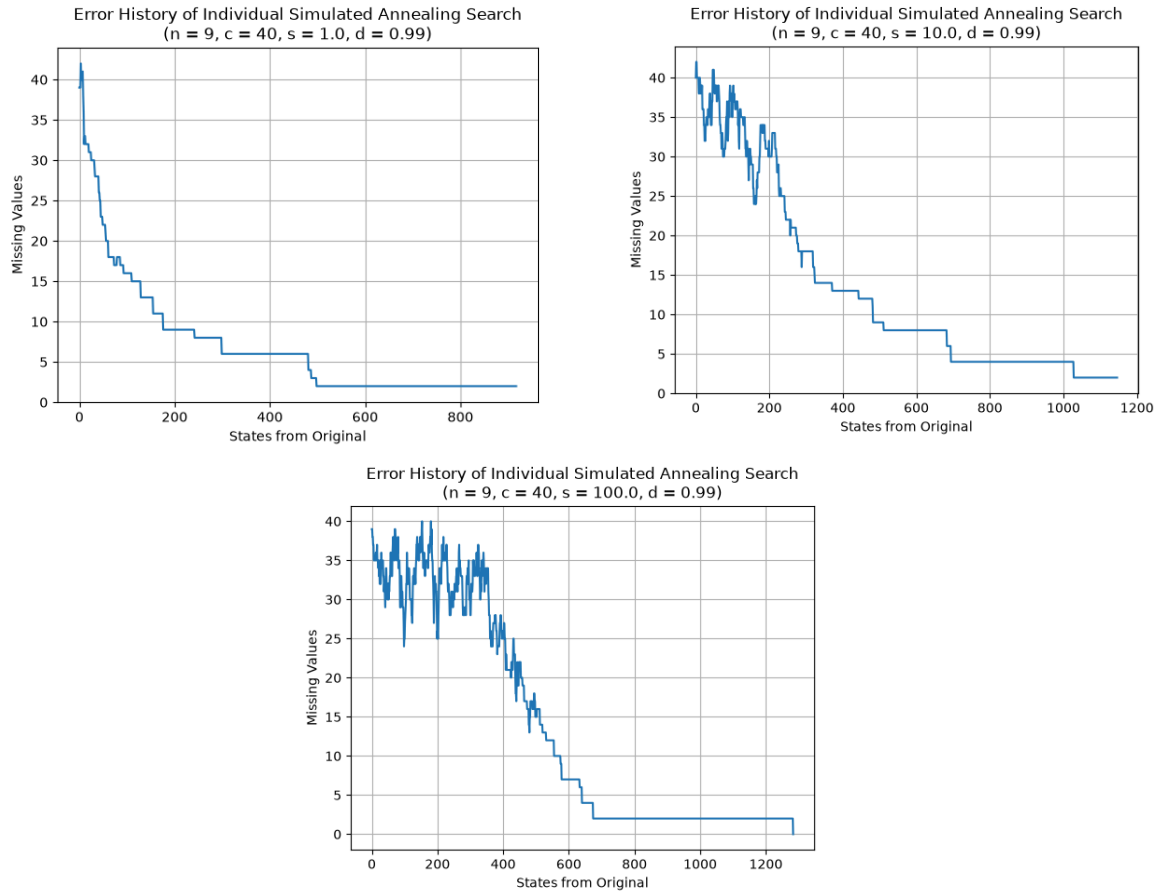


Figure 3.

3. While a higher exploration factor may lead to more incomplete Sudoku puzzles with higher error, a higher number of states visited counteracts this effect by lowering the final solution error, as stated in Question 1. I do not notice significant differences in the following histograms, although further statistical analysis is warranted.

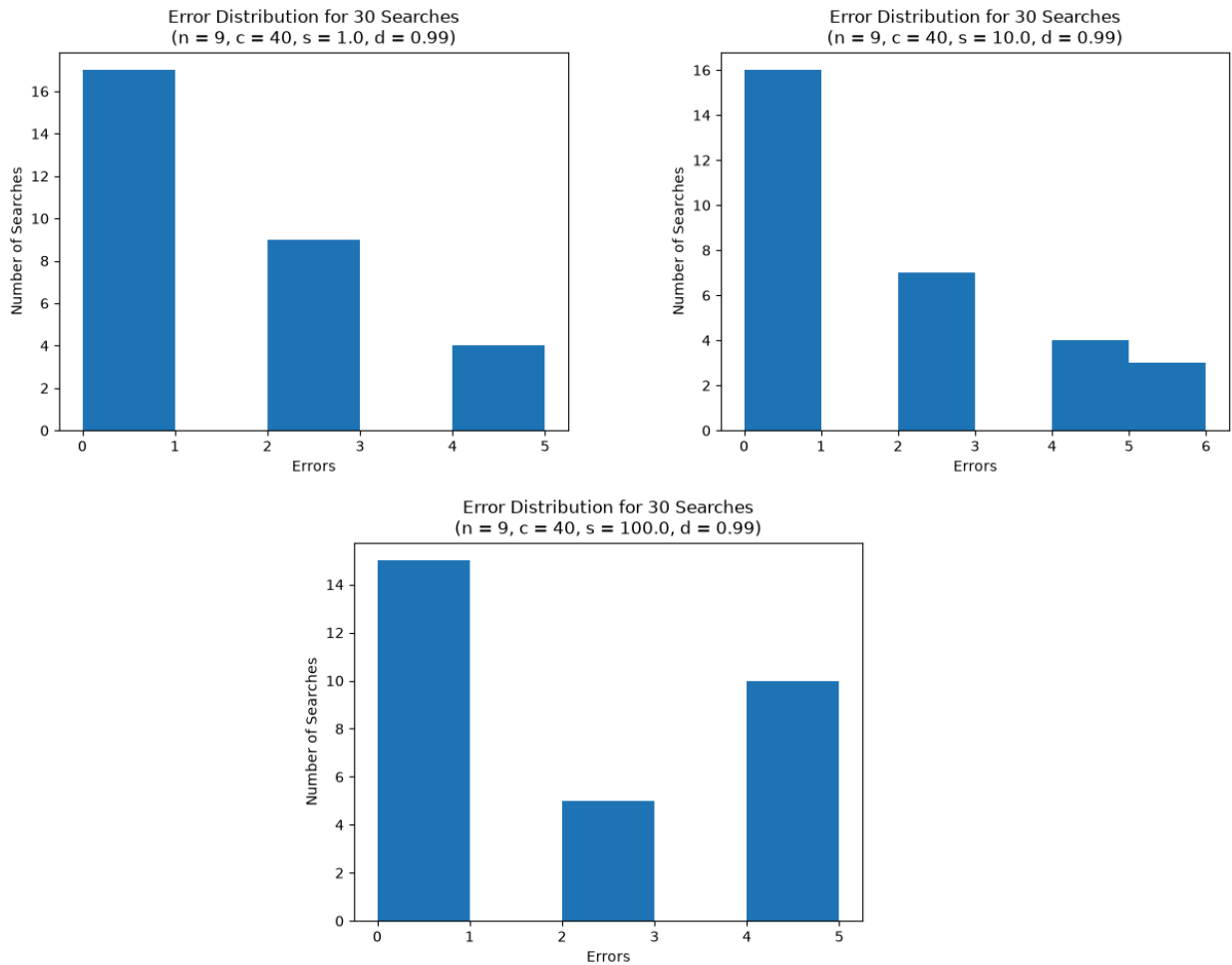


Figure 4.

Problem 5

1. See the corresponding figures below, as well as the command line output. Theoretically, DFS' space complexity should be linear with respect to the maximum depth and BFS' should be exponential, DFS's maximum frontier size was an order of magnitude larger than DFS' (5082 versus 147). This discrepancy comes from DFS's pathfinding, which heat-coils around the entire map before reaching the goal, leading it to expand significantly more states on its solutions path compared to BFS. Additionally, path pruning means that a large number of child states in BFS become pruned, keeping its frontier size low.

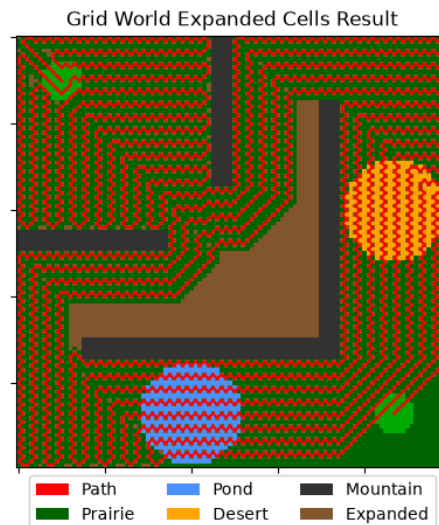


Figure 5. DFS results for World 1.

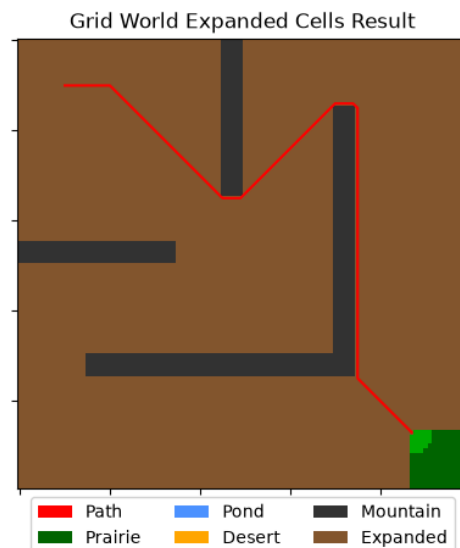


Figure 6. BFS results for World 1.

Python

Testing Grid World Path Planning...

Loading grid world file from path: worlds

Testing world 1

Modes: 1. DFS, 2. BFS

Mode: DFS

Path length: 2583

Path cost: 2655.0

Number of expanded states: 3665

Max frontier size: 5082

Mode: BFS

Path length: 138

Path cost: 138

Number of expanded states: 8907

Max frontier size: 147

Done

2. See figures below. As expected, A* beam search has a lower maximum frontier size than regular A* (the default 50 versus 555), but A* returns a more optimal solution (path cost of 75 versus 77). Beam search sacrifices solution optimality for lower space complexity. When using the Euclidean heuristic instead of the Manhattan heuristic, both A* and A* beam search expands significantly more states (5122 and 2118 respectively, compared to 1281 and 540). However, the expansions occurred along diagonal paths, which mirrored the optimal solution paths, which allowed both search algorithms to reach a more optimal solution (path cost of 74.5 for both).

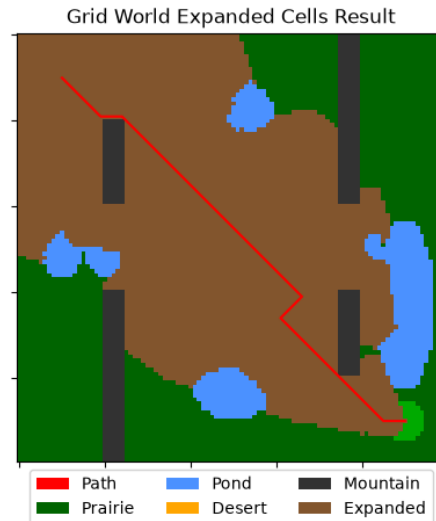


Figure 7. A* result on World 2 with Manhattan heuristic.

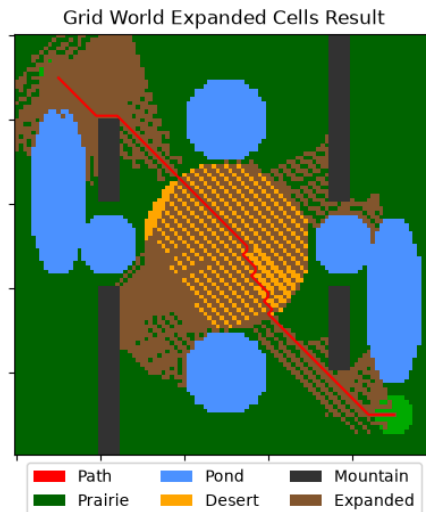


Figure 8. A* beam search's results on World 2 with Manhattan heuristic.

3. See figure below for A* beam search (A* generates identical results to Figure 7). When using the Manhattan heuristic, setting the width to 100 allows A* beam search to explore into the cheaper-cost desert region and discover the same path as A* (path cost of 75). However, the number of expanded states increases accordingly (from 540 to 827). While the theoretical upper limit for space complexity doubles when width doubles, path pruning may have eliminated a significant amount of reached states when the width increases, leading to lower number of expanded states.

Figure 9. A^* beam search on World 2 with Manhattan heuristic and width set to 100.